Module 5 Tangent: Type Systems

Generics

Also known by the cooler name “parametric polymorphism”

1.

Generics

Functions on data? Functions on functions? Functions on types!

Generics

Definition

Generics (parametric polymorphism) allows abstracting over types.

- Generics support classes/functions:
  - `List<T>; <N> N max(N, N)`
- Generics can also have **bounds**!
  - `<T:Comparable<T>> sort(...)`
- Notably common use cases:
  - Data Containers: Optional, List
  - Algorithm Abst.: .map(), .visit()

Implementation Approach

At runtime, generics can be handled via:

- Templates (C++): The compiler creates a new function for every generic type.
  - More optimizable, but more code.
- Type Erasure (Java): The compiler "erases" generics at runtime.
  - Generics are replaced with bounds (e.g. Object) and are lost at runtime.
  - Reified: Generics that aren't erased
    - Not in Java (caveat-ish: Arrays).

```java
//Compile-time (static) behavior:

Map<String, Integer> map
  = new HashMap<String, Integer>();


map.get("key"); //null
map.put("key", 1);
map.get("key"); //Integer 1


map.put("key", 2.0); //error!
map.get("key"); //Integer 1
```

```java
//Runtime (dynamic) behavior:

Map map = new HashMap();


//Map.get returns Object at runtime!
(Integer) map.get("key"); //null
map.put("key", 1);
(Integer) map.get("key"); //Integer 1

//Map.put will accept any Object
map.put("key", 2.0); //unsafe!
(Integer) map.get("key"); //CCE!

//The generics <String, Integer> are
//completely erased at runtime!
```

Generic Type Erasure (Java)

How do we typecheck generics?

Variable types need type variables!

```
add(List<T> list, T element);

add(List<Int>, Int);
// List<Int> <: List<T>, {}
//   - T = Int
// Int <: T, {T = Int}

add(List<Int>, Float);
// List<Int> <: List<T>, {}
//    T = Int
// Float <: T, {T = Int}
```

```
// What about subtyping…?

add(List<Num>, Int);
// List<Num> <: List<T>, {}
//   - T = Num
// Int <: T, {T = Num}

flippedAdd(T element, List<T> list);
flippedAdd(Int, List<Num>);
// Int <: T, {}
//   - {T = Int}
// List<Num> <: List<T>, {T = Int}
//   - error!

// We need variant constraints.
```

Generic Type Variables

2.

Variance

As if generics weren't complicated enough…

Variance

Definition

Variance refers to the subtyping constraints applied to generic bounds.

- **Covariant**: Maintains subtyping
- Contravariant: Inverts subtyping
- Invariant: Incompatible subtyping
  - `List<String>, List<Object>`
- In Java, Variance uses wildcards:
  - `List<? extends Number>`
  - `Allows .get(), not .set()`

Array Variance

In Java, Arrays are covariant at compile time but invariant at runtime.

- Conceptually, arrays are invariant
  - String[] is not a subtype of Object[]
  - Object[] is not a subtype of String[]
- However, for historical reasons arrays are Covariant within the compiler.
  - Storing a value of an incorrect type in an array will throw an exception.

```java
//Compile-time (static) behavior:

String[] strings = new String[3];

strings[0] = null;
strings[1] = "string";
strings[2] = 1; //error!

//Array types are covariant
Object[] objects = strings; //!!!

objects[2] = 1; //wat
strings[2].toUpperCase(); //???
```

```java
//Runtime (dynamic) behavior:

String[] strings = new String[3];

strings[0] = null;
strings[1] = "string";
strings[2] = 1; //error!

//Array types are covariant
Object[] objects = strings; //!!!

objects[2] = 1; //ArrayStoreException
strings[2].toUpperCase(); //NPE!
```

Covaraint Arrays (Java)

```
add(List<T> element, T element);

add(List<Int>, Int);
// List<Int> <: List<T>, {}
//   - T = Int
// Int <: T, {T = Int}

add(List<Int>, Float);
// List<Int> <: List<T>, {}
//    T = Int
// Float <: T, {T = Int}
```

```
// What about subtyping…?

add(List<Num>, Int);
// List<Num> <: List<T>, {}
//   - T = Num (invariant)
// Integer <: T, {T = Num}

flippedAdd(T element, List<T> list);
flippedAdd(Int, List<Num>);
// Int <: T, {}
//   - T >: Int (covariant)
// List<Num> <: List<T>, {T >: Int}
//   - T = Num (invariant)

// Problem solved! Mostly…
```

Generic Type Variables (Variant Constraints)

Module 5 Tangent: Type Systems

Generics

I am once again asking for you to take a quiz…

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals